

Absolute Visual Localization

Code Architecture & Developer Guide

Image-based Localization (GPS-Denied)

June 2026

Absolute Visual Localization

Code Architecture & Developer Guide

Image-based Localization (GPS-Denied)

June 2026

Encode images → FAISS search → GPS fusion
State-of-the-art MixVPR + approximate nearest-neighbor hashing

Contents

1	Introduction	2
2	System Architecture	2
2.1	High-Level Overview	2
2.2	Data Flow Diagram	2
2.3	Project Structure	2
3	Module Reference	3
3.1	AVLLocalizer — The Main Entry Point	3
3.1.1	Building an index (offline)	3
3.1.2	Localizing a query (online)	3
3.2	VPREncoder — Image Fingerprints	4
3.2.1	Supported models	4
3.2.2	Encoding pipeline	4
3.2.3	MixVPR internals	4
3.3	HashIndex — FAISS Similarity Search	5
3.3.1	Index types	5
3.3.2	Search	5
3.3.3	Persistence	5
3.4	geo.py — Geographic Math	6
3.4.1	Haversine distance	6
3.4.2	Weighted geo fusion	6
3.4.3	Confidence formula	6
3.5	metadata.py — Reference Database Format	6
3.5.1	Required CSV columns	6
3.5.2	Optional columns	7
3.6	AVLConfig — Configuration	7
4	Command-Line Tools	7
4.1	Build index	7
4.2	Localize query	7
5	Python API Example	8
6	Design Decisions & Limitations	8
6.1	What this system does well	8
6.2	Current limitations	9
6.3	Recommended extensions for production UAV use	9
7	Glossary	9
8	References	9

1 Introduction

This document explains the **Absolute Visual Localization (AVL)** codebase: a GPS-denied UAV localization system that matches live camera images against a database of **geo-tagged reference images**.

What problem does this solve? When GPS is unavailable or unreliable, a UAV can still estimate its global position by asking: *“Which known place does my camera see right now?”* Each reference image carries a known latitude/longitude. The closest visual match provides the location estimate.

Core pipeline in one sentence:

Encode images into vectors → search similar vectors with FAISS → fuse GPS coordinates of top matches.

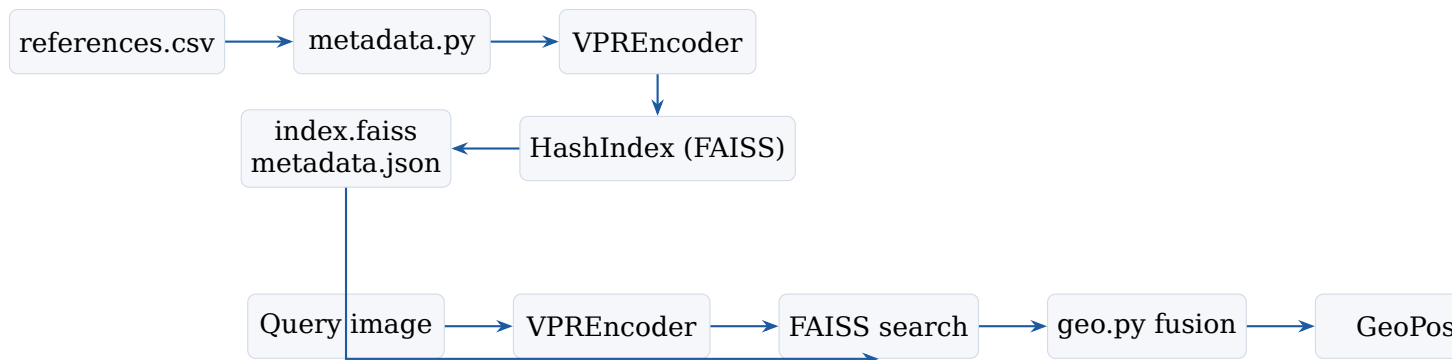
2 System Architecture

2.1 High-Level Overview

The system has two phases:

Phase	When	Input	Output
Index build (offline)	Once per map	Geo-tagged reference CSV + images	index.faiss + metadata.json
Localization (online)	Per UAV frame	Query image	lat, lon, altitude, confidence

2.2 Data Flow Diagram



2.3 Project Structure

Image-based-localization-GPS-Denied/

└─ avl/	# Core library
└─ config.py	# All tunable parameters
└─ encoder.py	# Neural image → descriptor
└─ index.py	# FAISS approximate search
└─ geo.py	# GPS math & fusion
└─ localizer.py	# Main orchestrator
└─ metadata.py	# CSV loading
└─ models/	
└─ mixvpr.py	# MixVPR network (default)
└─ cosplace.py	# CosPlace alternative
└─ scripts/	
└─ build_index.py	# CLI: build FAISS index
└─ localize.py	# CLI: localize query image
└─ create_examples.py	# Synthetic demo data
└─ examples/	
└─ references.csv	# Sample metadata format

3 Module Reference

3.1 AVLLocalizer — The Main Entry Point

File: `avl/localizer.py`

This class orchestrates the entire pipeline. It owns a VPREncoder and a HashIndex.

3.1.1 Building an index (offline)

```
def build_index(self, metadata_csv, output_dir, base_dir=None):
    records = load_reference_metadata(metadata_csv, base_dir=base_dir)
    image_paths = [record.image_path for record in records]
    descriptors = self.encoder.encode_paths(image_paths)
    self.index.build(descriptors, records)
    self.index.save(output_dir)
```

Step-by-step:

1. Parse the CSV into ReferenceRecord objects (path + GPS).
2. Encode every reference image into a descriptor matrix of shape (N, D).
3. Insert descriptors into FAISS and attach GPS metadata.
4. Persist to disk for later reuse.

3.1.2 Localizing a query (online)

```
def localize(self, query_image, top_k=None):
    descriptor = self.encoder.encode_image(query_image)
    search = self.index.search(descriptor, top_k=top_k)
    # ... build Match list from search results ...
    pose = weighted_geo_fusion(latitudes, longitudes, weights)
```

```
confidence = best.score * exp(-spread_m / 500)
return LocalizationResult(...)
```

Step-by-step:

1. Encode the UAV query frame.
2. Retrieve top- K most similar reference descriptors.
3. Reject if best score $<$ score_threshold (default 0.35).
4. Fuse GPS of top- K matches (weighted by similarity).
5. Compute confidence from match quality and geographic agreement.

Key data classes **Match** — one retrieved reference: rank, cosine score, GPS record.

LocalizationResult — final pose, confidence, best match, all matches, query descriptor.

3.2 VPREncoder — Image Fingerprints

File: `avl/encoder.py`

Converts RGB images into fixed-size **descriptor vectors** using a Visual Place Recognition (VPR) neural network.

3.2.1 Supported models

Model	Default dim	Input size	Source
MixVPR	4096	320×320	Pretrained weights via gdown
CosPlace	2048	322×322	PyTorch Hub (gmberton/CosPlace)

3.2.2 Encoding pipeline

```
# For each image batch:
batch = transform(image)           # resize + normalize
outputs = model(batch)             # neural network forward pass
outputs = F.normalize(outputs)     # L2 normalize → unit vector
```

After L2 normalization, the **dot product** between two descriptors equals **cosine similarity** (range roughly -1 to 1 ; higher = more similar places).

Why normalization matters FAISS is configured with METRIC_INNER_PRODUCT. This only equals cosine similarity when all vectors are unit-length. Both the encoder and index normalize vectors before search.

3.2.3 MixVPR internals

File: `avl/models/mixvpr.py`

MixVPR (WACV 2023) uses two components:

1. **ResNet50 backbone** (layers 1-3 only) — extracts spatial feature maps.
2. **MixVPR aggregator** — mixes feature-map relationships with MLP blocks, then projects to a compact global descriptor.

Image (320×320)

- ResNet50 (conv1 ... layer3)
- Feature maps (1024 × 20 × 20)
- FeatureMixer layers (×4)
- Channel + row projections
- L2-normalized descriptor (4096-dim)

Weights are downloaded once to ~/.cache/avl/mixvpr/.

3.3 HashIndex — FAISS Similarity Search

File: `avl/index.py`

FAISS (**F**acebook **A**I **S**imilarity **S**earch) finds the nearest descriptor vectors without comparing against every reference.

3.3.1 Index types

Type	Class	Speed	Accuracy	Use case
flat	IndexFlatIP	Slowest	Exact	Debugging, small maps
hnsw	IndexHNSWFlat	Fast	Near-exact	Default — up to ~1M images
ivfpq	IndexIVFPQ	Very fast	Approximate	Millions of tiles, low memory

HNSW builds a navigable graph over vectors. Search follows graph edges to quickly reach the nearest neighbors.

IVF-PQ clusters vectors (Inverted File) and stores compressed Product Quantization codes — this is the “hashing” approach for massive databases.

3.3.2 Search

```
faiss.normalize_L2(query)
scores, indices = index.search(query, top_k)
```

Returns the indices of the K closest references and their similarity scores.

3.3.3 Persistence

File	Contents
<code>index.faiss</code>	FAISS binary index (vectors + search structure)

File	Contents
metadata.json	Model config + GPS record per indexed image

3.4 geo.py — Geographic Math

File: `avl/geo.py`

3.4.1 Haversine distance

Computes great-circle distance in meters between two GPS points on the Earth's surface. Used to measure how spread out the top- K matches are (for confidence).

3.4.2 Weighted geo fusion

Instead of trusting only the single best match, the system fuses top- K GPS coordinates weighted by similarity score.

Critical detail: latitude and longitude are **not** averaged directly. They are converted to 3D unit-sphere coordinates (x, y, z) , weighted-averaged, then converted back:

```
x = cos(lat) * cos(lon)
y = cos(lat) * sin(lon)
z = sin(lat)
# weighted mean of x, y, z → back to lat/lon
```

This avoids errors near the poles and the $\pm 180^\circ$ longitude wrap.

3.4.3 Confidence formula

$$\text{confidence} = \text{best_score} \times e^{-\text{spread_m}/500}$$

- High best score $\rightarrow$ confident.
- Top matches agree geographically (low spread) $\rightarrow$ confident.
- Matches scattered far apart $\rightarrow$ confidence drops.

3.5 metadata.py — Reference Database Format

File: `avl/metadata.py`

3.5.1 Required CSV columns

Column	Type	Description
image_path	string	Path to reference image
latitude	float	WGS84 latitude (degrees)
longitude	float	WGS84 longitude (degrees)

3.5.2 Optional columns

Column	Description
altitude_m	Altitude in meters
heading_deg	Camera heading
image_id	Custom identifier

Example:

```
image_path,latitude,longitude,altitude_m,heading_deg,image_id
data/refs/tile001.jpg,48.8584,2.2945,50.0,180.0,tile001
```

3.6 AVLConfig — Configuration

File: `avl/config.py`

Parameter	Default	Meaning
model	mixvpr	VPR backbone (mixvpr or cosplace)
descriptor_dim	4096	Vector size
index_type	hns	FAISS index algorithm
top_k	5	Matches used for geo fusion
score_threshold	0.35	Minimum similarity to accept
device	cuda	PyTorch device (falls back to CPU)
batch_size	16	Encoding batch size

4 Command-Line Tools

4.1 Build index

```
python scripts/build_index.py \
  --metadata data/references.csv \
  --base-dir data/ \
  --output artifacts/uav_map_index \
  --model mixvpr \
  --descriptor-dim 4096 \
  --index-type hns
```

4.2 Localize query

```
python scripts/localize.py \
  --index artifacts/uav_map_index \
  --query path/to/uav_frame.jpg \
  --top-k 5 \
  --json-out outputs/result.json
```

JSON output fields:

Field	Description
latitude, longitude	Fused position estimate
altitude_m	Weighted altitude (if available)
confidence	Quality score (0-1 range)
best_match	Top-1 reference details
matches	Full top- K list with scores

5 Python API Example

```

from pathlib import Path
from avl import AVLLocalizer
from avl.config import AVLConfig

config = AVLConfig(model="mixvpr", descriptor_dim=4096, index_type="hnsw")
localizer = AVLLocalizer(config)

# Offline: build once
localizer.build_index(
    metadata_csv=Path("data/references.csv"),
    output_dir=Path("artifacts/index"),
    base_dir=Path("data"),
)

# Online: per frame
localizer.load_index(Path("artifacts/index"))
result = localizer.localize(Path("query.jpg"))

print(f"Position: {result.pose.latitude:.6f}, {result.pose.longitude:.6f}")
print(f"Confidence: {result.confidence:.3f}")
print(f"Best match: {result.best_match.record.image_path}")

```

6 Design Decisions & Limitations

6.1 What this system does well

- Fast retrieval over large geo-tagged image databases.
- State-of-the-art VPR descriptors (MixVPR).
- Configurable FAISS indexes for speed vs. scale.
- Robust GPS fusion from multiple matches.
- Simple CSV-based reference format.

6.2 Current limitations

Not included in this version

- **No keypoint matching** — no SIFT/SuperPoint geometric verification.
- **No temporal filtering** — single-frame only; no video smoothing.
- **No GPS prior** — search is global; no bounding-box restriction.
- **Cross-view gap** — nadir UAV vs. oblique satellite may need specialized models.

6.3 Recommended extensions for production UAV use

1. **Geographic pre-filter** — restrict search to a tile around last known GPS.
2. **Multi-frame voting** — median or Kalman filter over consecutive frames.
3. **Local feature verification** — SuperPoint + LightGlue on top- K candidates.
4. **Altitude aiding** — barometer + homography scale consistency check.

7 Glossary

Term	Definition
AVL	Absolute Visual Localization — global lat/lon from images
VPR	Visual Place Recognition — matching places by appearance
Descriptor	Fixed-size numeric fingerprint of an image
FAISS	Library for fast approximate nearest-neighbor search
HNSW	Hierarchical Navigable Small World graph index
IVF-PQ	Inverted File + Product Quantization (compressed hashing)
Cosine similarity	Measure of vector similarity (-1 to 1)
Top-K fusion	Combining K best matches into one GPS estimate
Geo-tagged	Image with known GPS coordinates attached

8 References

- MixVPR: Ali-bey et al., WACV 2023 — <https://arxiv.org/abs/2303.02190>
- CosPlace: Berton et al., CVPR 2022 — <https://arxiv.org/abs/2204.02287>
- FAISS: Johnson et al. — <https://github.com/facebookresearch/faiss>